

Parte I — wait/notify: Productor/Consumidor

1. Ejecuta el programa de productor/consumidor y monitorea CPU con jVisualVM.

¿Por qué el consumo alto? ¿Qué clase lo causa?

El alto consumo de CPU se debe a que el sistema usa espera activa (busy-waiting) en lugar de bloqueo de hilos. En espera activa, los hilos no liberan la CPU cuando no pueden progresar, sino que ejecutan bucles infinitos verificando repetidamente una condición compartida.

En el escenario donde **el productor es lento y el consumidor es rápido**, el consumidor intenta extraer elementos constantemente cuando la cola está vacía. En lugar de dormir, entra en un ciclo infinito verificando si hay datos disponibles.

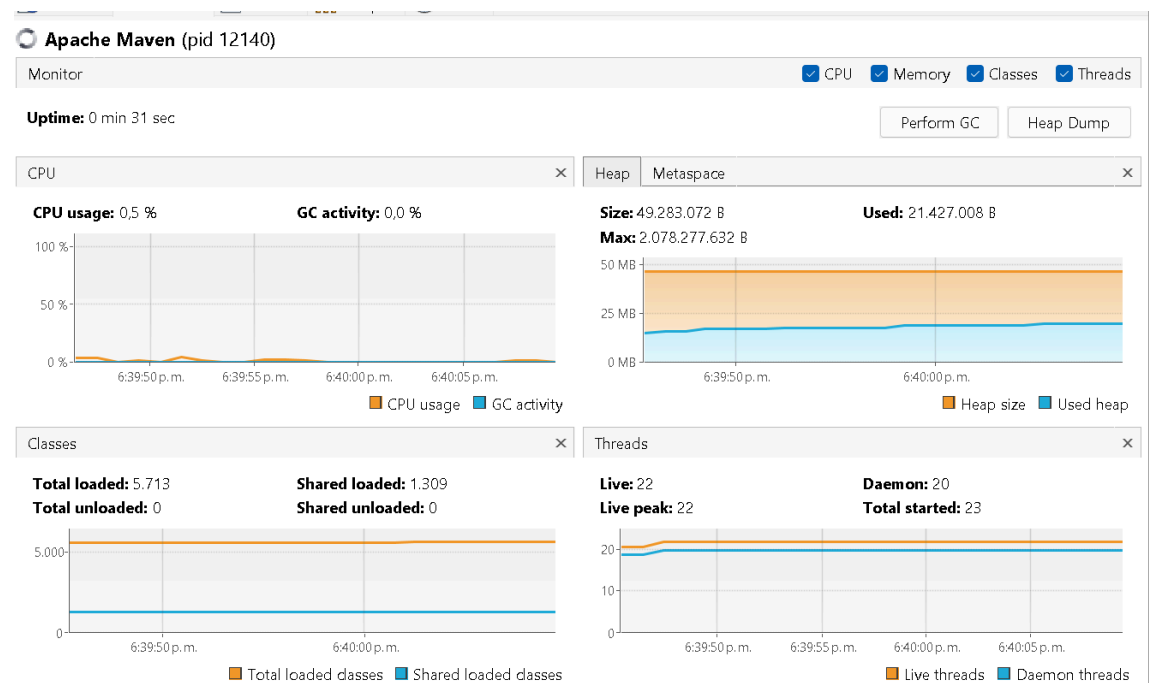
La clase responsable es **BusySpinQueue** (Método **T take()**) que básicamente el hilo nunca se bloquea, solo gira consumiendo CPU, aun cuando no hay trabajo real que hacer

En este caso, los hilos productores y consumidores permanecen en estado **RUNNABLE**, ejecutando instrucciones de usuario de manera continua, aun cuando la cola está llena o vacía.

2. Ajusta la implementación para usar CPU eficientemente cuando el productor es lento y el consumidor es rápido. Valida de nuevo con VisualVM.

Se usó la clase de referencia **BoundedBuffer** para reemplazar la espera activa por monitores usando **synchronized**, **wait**, **notifyAll**.

Ejecutar con monitores (uso eficiente de CPU)



CPU

- Uso de CPU $\approx 0.5\%$
- Línea estable, sin picos
- No hay consumo constante

Esto indica que los hilos NO están girando activamente.

Threads

- Threads vivos: 22
- Sin crecimiento descontrolado
- Estados mayoritariamente WAITING

Productor o consumidor duermen cuando no pueden avanzar.

Memoria / GC

- Heap estable (21 MB usados)
- GC activity: 0%

No hay presión de memoria ni creación excesiva de objetos.

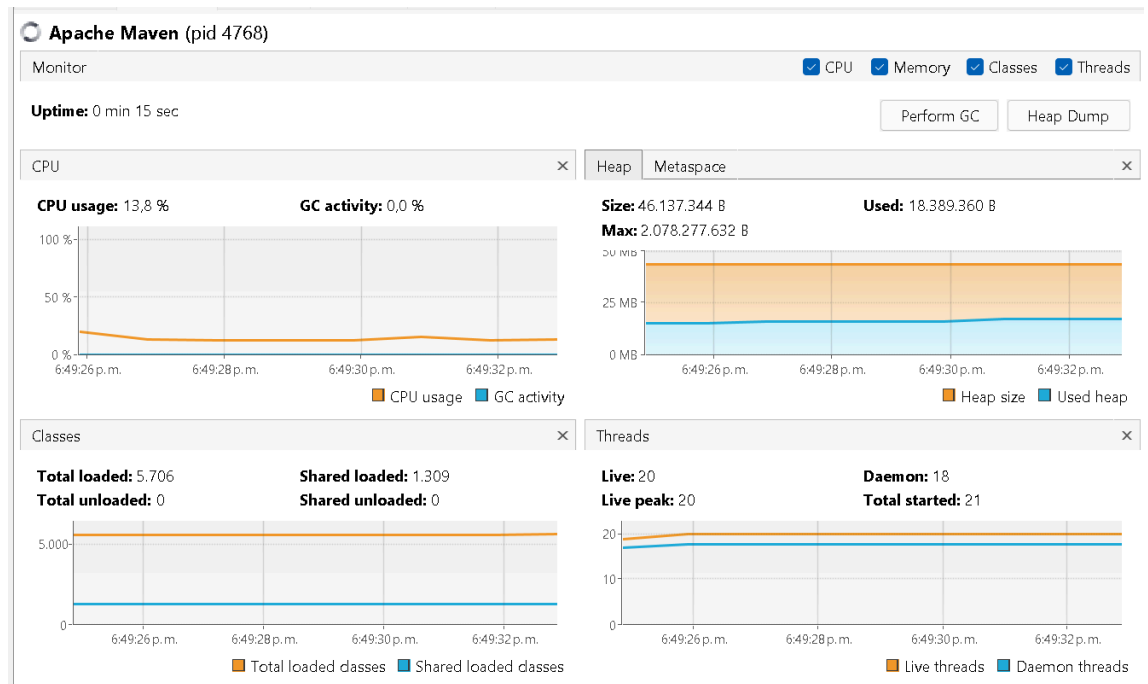
En el caso cuando la cola está vacía el consumidor libera el lock, entra en estado WAITING y no consume CPU.

Cuando el productor agrega un elemento llama a notifyAll y despierta al consumidor

3. Ahora productor rápido y consumidor lento con límite de stock (cola acotada): garantiza que el límite se respete sin espera activa y valida CPU con un stock pequeño.

En este escenario El productor genera datos más rápido de lo que el consumidor puede procesar y la cola tiene un límite fijo

Ejecutar con busy-wait (alto CPU)



CPU

- Uso de CPU $\approx$ 13–15% constante
- No hay caídas a 0
- Actividad continua aunque no haya trabajo real

Esto es típico de **busy-wait**: el hilo **gira todo el tiempo**.

Threads

- 20 hilos vivos
- Estado principalmente **RUNNABLE**
- No entran en **WAITING**

El consumidor **no duerme**, sigue intentando consumir.

Memoria / GC

- Heap estable (~18 MB usados)
- GC activity: **0%**

El problema **no es memoria**, es **CPU**.